Part 4 — Docker Deep Dive

Probability: 🔴 95% — Very Frequently Asked

Dockerfile — Line by Line

```
# === Stage 1: Build ===
FROM node:18-alpine AS builder        # 1. Base image (alpine = minimal)
# Q: "Why alpine?" → Smaller image (5MB vs 900MB), less attack surface
# TRAP: "But alpine uses musl libc, not glibc — some native modules may break"

WORKDIR /app                          # 2. Set working directory
# Creates dir if not exists. All subsequent commands run from here

COPY package*.json ./                 # 3. Copy dependency files FIRST
# Q: "Why copy package.json separately?" → Docker layer caching!
# If source code changes but deps don't, this layer is cached

RUN npm ci --only=production          # 4. Install dependencies
# Q: "npm ci vs npm install?" → ci uses lockfile exactly, faster, deterministic
# This layer is cached if package.json unchanged

COPY . .                              # 5. Copy source code
RUN npm run build                     # 6. Build application

# === Stage 2: Production ===
FROM node:18-alpine AS production     # 7. Fresh base — no build tools
# MULTI-STAGE: Only the final stage becomes the image

RUN addgroup -S appgroup && \
    adduser -S appuser -G appgroup    # 8. Non-root user
# SECURITY: Never run as root in production

WORKDIR /app

COPY --from=builder /app/dist ./dist           # 9. Copy ONLY build output
COPY --from=builder /app/node_modules ./node_modules
COPY --from=builder /app/package.json ./
# Result: No source code, no devDependencies, no build tools in final image

EXPOSE 3000                           # 10. Document the port
# Q: "Does EXPOSE open the port?" → NO. It's documentation only.
# You still need -p 3000:3000 at runtime

HEALTHCHECK --interval=30s --timeout=3s \
  CMD wget --spider http://localhost:3000/health || exit 1
# 11. Container health monitoring

USER appuser                          # 12. Switch to non-root
```

```
ENTRYPOINT ["node"]                    # 13. Fixed executable
CMD ["dist/server.js"]                # 14. Default arguments
```

ENTRYPOINT vs CMD — The Most Asked Docker Question

```
ENTRYPOINT = The executable (fixed)
CMD        = Default arguments (overridable)

Combined: ENTRYPOINT + CMD = full command

Example:
ENTRYPOINT ["node"]
CMD ["server.js"]
→ Runs: node server.js

Override CMD: docker run myimage worker.js
→ Runs: node worker.js

Override ENTRYPOINT: docker run --entrypoint python myimage script.py
→ Runs: python script.py
```

Instruction	Purpose	Overridable?
`ENTRYPOINT`	Defines the main executable	Only with `--entrypoint` flag
`CMD`	Default arguments to ENTRYPOINT	Yes, by appending args to `docker run`
Only `CMD`	Entire command + args	Yes, fully replaceable

Q: "When do you use ENTRYPOINT vs CMD?"

> *"Use ENTRYPOINT when the container is a specific tool (always runs node, or nginx). Use CMD for default behavior that users might override. For production microservices: ENTRYPOINT for the runtime + CMD for the script. This allows overriding the script for debugging without changing the runtime."*

Multi-Stage Build — Real Production Example

```
# Stage 1: Dependencies (cached heavily)
FROM golang:1.21-alpine AS deps
WORKDIR /app
COPY go.mod go.sum ./
RUN go mod download                    # Cached if go.mod unchanged

# Stage 2: Build
FROM deps AS builder
COPY . .
RUN CGO_ENABLED=0 GOOS=linux go build -ldflags="-s -w" -o /app/server ./cmd/server
#    ^^^^^^^^^^^^                ^^^^^^^^^^^^^^^^^
#    Static binary               Strip debug info (smaller binary)
```

```
# Stage 3: Production (FROM SCRATCH = empty image)
FROM scratch                          # 0 bytes base image!
COPY --from=builder /app/server /server
COPY --from=builder /etc/ssl/certs/ca-certificates.crt /etc/ssl/certs/
EXPOSE 8080
ENTRYPOINT ["/server"]
# Final image: ~10MB vs ~1GB with full golang base
```

Q: "What is a multi-stage build and why use it?"

> *"Multi-stage builds use multiple FROM statements. Each FROM starts a new stage. Only the final stage becomes the image. Benefits: 1) Build tools (compilers, dev deps) stay in build stage — not shipped to production. 2) Smaller images (we reduced from ~1GB to ~15MB). 3) Reduced attack surface — fewer packages = fewer CVEs. 4) Faster deployments — smaller images pull faster."*

Docker Compose

```
# docker-compose.yml
version: '3.8'

services:
  app:
    build:
      context: .
      dockerfile: Dockerfile
      target: production              # Build specific stage
    ports:
      - "3000:3000"
    environment:
      - NODE_ENV=production
      - DB_HOST=postgres
    depends_on:
      postgres:
        condition: service_healthy    # Wait for health check
    networks:
      - app-network
    restart: unless-stopped
    deploy:
      resources:
        limits:
          memory: 512M
          cpus: '0.5'

  postgres:
    image: postgres:15-alpine
    volumes:
      - pgdata:/var/lib/postgresql/data  # Named volume (persistent)
    environment:
      POSTGRES_DB: myapp
```

```
      POSTGRES_PASSWORD_FILE: /run/secrets/db_password  # Docker secrets
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 10s
      timeout: 5s
      retries: 5
    networks:
      - app-network

  redis:
    image: redis:7-alpine
    command: redis-server --requirepass ${REDIS_PASSWORD}
    networks:
      - app-network

volumes:
  pgdata:                              # Persists across container restarts

networks:
  app-network:
    driver: bridge
```

Docker Networking

```
NETWORK TYPES
┌─────────────────────────────────────────────┐
│ bridge (default)                            │
│ • Containers on same bridge can communicate │
│ • Isolated from host network                │
│ • Use for: single-host multi-container apps │
├─────────────────────────────────────────────┤
│ host                                        │
│ • Container shares host's network stack     │
│ • No port mapping needed                    │
│ • Use for: performance-critical apps        │
├─────────────────────────────────────────────┤
│ none                                        │
│ • No networking                             │
│ • Use for: batch jobs, security isolation   │
├─────────────────────────────────────────────┤
│ overlay                                     │
│ • Multi-host networking (Swarm/K8s)         │
│ • Use for: distributed applications         │
└─────────────────────────────────────────────┘
```

Q: "How do containers in the same Docker Compose communicate?"

> *"Docker Compose creates a default bridge network. Containers resolve each other by service name as hostname. So `app` can reach `postgres` at `postgres:5432`. No IP addresses needed. This is*

> *Docker's built-in DNS."*

Docker Volumes

Type	Syntax	Persistence	Use Case
Named Volume	`volumes: pgdata:/var/lib/data`	Persists after container removal	Database storage
Bind Mount	`volumes: ./src:/app/src`	Host directory	Development hot-reload
tmpfs	`tmpfs: /tmp`	Memory only, lost on stop	Sensitive temp data

Q: "Named volume vs bind mount?"

> *"Named volumes are managed by Docker — portable, backed up easily, work on all platforms. Bind mounts map a host directory directly — great for development but not portable. In production, always use named volumes. In development, bind mounts enable hot-reload."*

Image Optimization Best Practices

Practice	Impact	Example
Use alpine/slim bases	5MB vs 900MB	`FROM node:18-alpine`
Multi-stage builds	Remove build tools	See example above
Order layers by change frequency	Better caching	COPY deps → install → COPY source
Combine RUN commands	Fewer layers	`RUN apt-get update && apt-get install -y curl && rm -rf /var/lib/apt/lists/*`
Use .dockerignore	Smaller build context	Ignore `node_modules`, `.git`, `*.md`
Pin image versions	Reproducibility	`FROM node:18.17.1-alpine` not `FROM node:latest`
Don't install unnecessary packages	Smaller image, fewer CVEs	`--no-install-recommends`
Use `COPY` not `ADD`	Predictable behavior	`ADD` has extra features (URL fetch, tar extract) you rarely need

.dockerignore

```
node_modules
.git
.env
*.md
docker-compose*.yml
Dockerfile
.dockerignore
coverage/
tests/
```

Security Best Practices

1. **Never run as root** — `USER nonroot` or `USER 1000`
2. **Scan images** — `grype myimage:tag`, `trivy image myimage:tag`
3. **Pin base image digests** — `FROM node:18-alpine@sha256:abc123...`
4. **No secrets in images** — Use runtime env vars or mounted secrets
5. **Read-only filesystem** — `docker run --read-only`
6. **Drop capabilities** — `docker run --cap-drop=ALL --cap-add=NET_BIND_SERVICE`
7. **Use distroless/scratch** — Minimal attack surface for compiled languages

---

Top 30 Docker Interview Questions

#	Question	Key Points
1	Difference between image and container?	Image = blueprint (read-only). Container = running instance
2	Docker layers — how do they work?	Each instruction creates a layer. Layers are cached and shared
3	COPY vs ADD?	COPY is simple copy. ADD can fetch URLs and extract tars. Use COPY
4	ENTRYPOINT vs CMD?	See detailed section above
5	What is a dangling image?	Untagged image, usually old build. `docker image prune` to clean
6	How to reduce image size?	Multi-stage, alpine, .dockerignore, combine RUN
7	Docker networking types?	bridge, host, none, overlay
8	What is Docker Compose?	Multi-container orchestration tool using YAML
9	How do you persist data?	Named volumes for production, bind mounts for dev
10	What is a health check?	Container-level health monitoring. Restarts if unhealthy
11	Docker vs Podman?	Podman is daemonless, rootless by default, OCI-compatible
12	What is a Docker registry?	Image storage. DockerHub (public), ACR/ECR/GitLab (private)
13	Explain layer caching	Unchanged layers are cached. Order matters — volatile last

#	Question	Key Points
14	How do you handle secrets?	Runtime env vars, Docker secrets, mounted volumes. Never in image
15	What is build context?	Directory sent to Docker daemon for build. Minimize with .dockerignore
16	Explain `docker system prune`	Removes stopped containers, unused networks, dangling images, build cache
17	What is a scratch image?	Empty base image — 0 bytes. For statically compiled binaries
18	How does Docker DNS work?	Built-in DNS server resolves container names on user-defined networks
19	Docker Swarm vs Kubernetes?	Swarm is simpler but less feature-rich. K8s is industry standard
20	What is `docker exec`?	Run a command in a running container. Debug tool
21	How to debug a crashed container?	`docker logs`, `docker inspect`, `docker cp` to extract files
22	What are cgroups and namespaces?	cgroups = resource limits. namespaces = isolation (PID, network, mount)
23	Multi-architecture builds?	`docker buildx build --platform linux/amd64,linux/arm64`
24	Docker rate limiting?	DockerHub limits pulls. Solution: use private registry or mirror
25	Container vs VM?	Container: process isolation, shared kernel, lightweight. VM: full OS
26	What is `docker commit`?	Create image from running container. Anti-pattern — use Dockerfiles
27	How to limit container resources?	`--memory=512m --cpus=0.5` or resource limits in compose
28	What is OCI?	Open Container Initiative — standard for container images and runtimes
29	How to handle logging?	Log to stdout/stderr. Collect with Fluent Bit/Fluentd. Don't log to files
30	Dockerfile best practices for CI/CD?	Cache deps layer, multi-stage, tag with Git SHA, scan in pipeline